

SPI-mforce Backend — AI-Powered Marketing Experiment Platform

Comprehensive Project Description for Resume

Project Overview

****spi-mforce-BE**** is a sophisticated backend system for an AI-powered marketing experiment platform that guides users through designing, configuring, and launching marketing experiments on Meta (MALA). The platform features a ****6-stage LangGraph agent orchestration pipeline**** with advanced multi-agent collaboration, MCP tool integration, and real-time conversational AI guidance.

****Core Value****: Automated end-to-end workflow for marketing experiment design with AI-driven recommendations, statistical power calculations, and KPI discovery.

Architecture & Services

Microservices Architecture

Three independently deployable services + shared database package:

| Service | Port | Technology Stack | Role |

|-----|-----|-----|-----|

| ****Agent Server**** | 8080 | Python 3.12, FastAPI, LangGraph, CopilotKit SDK | Main orchestrator; 6-stage agent workflow; CopilotKit protocol handler |

| **MCP Server** | 4001 | Python 3.12, FastMCP, FastAPI, PostgreSQL+PGVector | Model Context Protocol tools; campaign data retrieval; statistical calculations; RAG/semantic search |

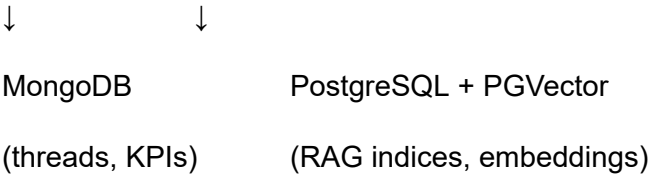
| **Copilot Runtime** | 4000 | Node.js 18+, Express, @copilotkit/runtime | Optional proxy layer; frontend integration |

| **Database Layer** | — | Python package (workspace dep) | Async MongoDB client; Thread service; Pydantic models |

Data Flow

...

Frontend (CopilotKit) → Agent Server (8080) → MCP Server (4001)



...

Core Technology Stack

Backend Framework & LLM Orchestration

- **LangGraph** (1.1.x–1.2.0): Multi-agent graph orchestration with checkpoint persistence
- **FastAPI** (0.115.14): High-performance async web framework for REST APIs
- **CopilotKit** (0.1.79+): Frontend-backend integration for real-time AI assistants
- **LangChain** (1.2.0+): LLM abstraction layer; structured output; tool binding
- **Pydantic** (2.5+): Data validation, type safety, structured output schemas

****LLM Providers****

- ****OpenAI**** (gpt-4, gpt-4-turbo, gpt-5): Primary LLM backend via `langchain-openai`
- ****Groq**** (Mixtral, Llama): Fallback provider via `langchain-groq`
- ****Response Streaming****: Streaming responses with JSON schema validation
- ****Structured Output****: Pydantic models with OpenAI structured output format (`json\_schema`, `function\_calling` modes)

****Model Context Protocol (MCP)****

- ****FastMCP**** (2.13.0.2): MCP server implementation
- ****20+ Tools****: Campaign data retrieval, KPI discovery, statistical power calculations, RAG search
- ****Tool Classes****:
 - `get\_kpi\_power\_context()`: Campaign metrics + statistical power calculations
 - `get\_campaign\_data()`: Raw Meta campaign analytics
 - `statistical\_power\_split\_test()`: A/B test power analysis
 - `conversion\_lift\_power()`: Conversion lift study calculations
 - `fetch\_mec\_kb\_PG()`: Semantic search via PostgreSQL + PGVector
 - Custom tools for campaign aggregation, data transformation, insights generation

****Databases****

- ****MongoDB**** (4.12.1): Persistent thread storage, experiment metadata, conversation history
- ****PostgreSQL**** (psycopg[pool] 3.2.7): PGVector embeddings for RAG, semantic search
- ****MongoDB TLS+Cert Auth****: Secure connection to DocumentDB in production

****Data & ML****

- **Pandas** (2.3.3): Data manipulation, campaign metrics aggregation
- **Scikit-learn** (1.8.0): Statistical analysis, power calculations
- **BM25** (rank-bm25 0.2.2): Hybrid search (sparse retrieval with vector embeddings)
- **OpenAI Embeddings** (text-embedding-3-large): Dense vector embeddings for semantic search
- **msgpack** (1.1.2): Efficient serialization for checkpoint storage

Observability & Tracing

- **Langfuse** (3.11.2): LLM tracing, prompt versioning, cost tracking
- **LangSmith** (0.7.22): LangChain debugging & monitoring
- **Custom Tracer**: `LangfuseTracer` with control-flow awareness for graph interrupts

Document Processing

- **pdfplumber** (0.11.9): PDF extraction
- **docx2txt** (0.9): Word document parsing
- **python-pptx** (1.0.2): PowerPoint processing
- **openpyxl** (3.1+): Excel spreadsheet handling

Graph Checkpointing

- **langgraph-checkpoint** (3.0.1+): Base checkpoint abstraction
- **langgraph-checkpoint-sqlite** (3.0.3+): Local development checkpointing
- **langgraph-checkpoint-postgres** (3.0.5+): Production-grade persistent checkpointing
- **aiosqlite** (0.21.0): Async SQLite for local development

AWS Integration

- **boto3** (1.40.70): AWS SDK for S3 artifact retrieval
- **S3 Storage**: Campaign indices, pre-computed embeddings, model artifacts

Multi-Agent Workflow (6 Stages)

Orchestrated via LangGraph with stage-specific state and subgraphs:

Stage 1: Prerequisite Selection

- **State**: Advertiser, ad account, country, campaign, budget, duration
- **Flow**: Form-based collection + validation

Stage 2: Objective Discovery

- **State**: Business goal + learning interest
- **Mechanism**: Conversational AI + MCP tools for campaign context
- **Tools**: Campaign data retrieval, historical insights

Stage 3: Test Type Suggestion

- **Recommendations**: A/B test, conversion lift, brand lift
- **Logic**: Business goal classification + statistical feasibility
- **Validation**: Test type suitability scoring

Stage 4: KPI Discovery & Selection

- ****State****: Selected KPI + Minimum Detectable Effect (MDE)

- ****Mechanism****: LLM agent with structured output

- KPI selection with human-readable descriptions
- Power analysis (sample size, duration calculations)
- Baseline rate collection for conversion lift

- ****Tools****:

- ``get_kpi_power_context()``: Real campaign data
- Statistical power calculations
- Hybrid semantic search (BM25 + vector embeddings)

****Stage 5: Lever Analysis & Campaign Optimization****

- ****State****: Campaign data analysis + experiment configuration recommendations

- ****Mechanism****:

- Fetch raw campaign metrics
- Transform & aggregate across dimensions (creative, audience, placement)
- Generate lever-specific recommendations
- Speed optimization via explicit checkpoint subgraph

- ****Tools****: Campaign data transformation, financial optimization, audience segmentation

****Stage 6: Experiment Design & MALA Push****

- ****State****: Finalized experiment blueprint

- ****Mechanism****:

- Consolidate all prior decisions
- Generate Meta MALA-compatible experiment spec

- Push to Meta platform via API

****Unified Router Logic****

- Conditional transitions between stages
- Skip logic for non-applicable stages (e.g., KPI not needed for CONVERSION_LIFT/BRAND_LIFT)
- Resumption support for interrupted threads

****Key Architectural Features****

****1. Conversational AI with Streaming****

- Real-time message streaming to frontend via CopilotKit protocol
- Graceful handling of mid-stream client disconnects
- Tool call rendering in browser
- State snapshots for UI synchronization

****2. State Management****

- ****CopilotKitState****: Base state with message history + custom fields
- ****Custom State Classes****: Stage-specific state (e.g., `KPIAgentState`, `LeverAgentState`)
- ****Blueprint Updater****: Automatic MongoDB sync of state changes
- ****Thread Persistence****: Full conversation + experiment state preserved across sessions

****3. Tool Integration****

- **MCP Client Cache**: TTL-based caching to avoid redundant tool calls
- **Tool Binding**: Dynamic MCP tool binding at runtime
- **Forced Tool Calls**: Completion gate logic enforces required MCP calls (e.g., power calculations)
- **Error Handling**: Graceful fallback when tools unavailable

4. Prompt Management

- **Langfuse Prompt Hub**: Centralized version control + A/B testing
- **Local Fallback**: Self-contained prompt files for offline operation
- **Dynamic Compilation**: System prompt + context injection at runtime
- **Context Shapes**:
 - XML-formatted workflow context
 - Campaign metrics + KPI data
 - Statistical power calculation inputs
 - Conversation history + collected values

5. Hybrid Search (KPI Discovery)

- **Sparse Retrieval**: BM25 on KPI metadata (sys_id, name, description)
- **Dense Retrieval**: OpenAI embeddings + cosine similarity
- **Reciprocal Rank Fusion (RRF)**: Combined scoring (BM25 0.4, vector 0.6)
- **Semantic Deduplication**: Avoid near-duplicate KPI suggestions

6. Statistical Power Analysis

- **MDE Extraction**: Minimum Detectable Effect from user intent
- **Sample Size Calculation**: Based on baseline, improvement target, confidence

- **Duration Estimation**: Given weekly impressions + sample size
- **Type-Specific Logic**:
 - **Split Test**: Volume-based (impressions per week)
 - **Conversion Lift**: Baseline rate + audience size
 - **Brand Lift**: Reach + frequency

7. Checkpointing & Resumption

- **Graph Checkpoints**: Node-level execution snapshots
- **Two Backends**:
 - SQLite (default): Local development
 - PostgreSQL: Production with multi-instance support
- **Thread-Scoped State**: Full history maintained across turns
- **Regeneration Support**: Time-travel replay from checkpoints

Code Quality & DevOps

Code Quality Tools

- **Ruff** (0.9+): Linting + formatting (100-char lines, double quotes)
- **mypy** (1.14+): Static type checking with Pydantic support
- **Pre-commit Hooks**:
 - Ruff (lint + format)
 - mypy

- Trailing whitespace
- End-of-file fixer
- YAML validation
- Large file detection

****Testing****

- ****pytest**** (8.0+): Test runner with asyncio support
- ****pytest-asyncio**** (0.24+): Async/await test execution
- ****pytest-cov**** (6.0+): Coverage reporting
- ****pytest-mock**** (3.14+): Mocking utilities
- ****Markers****: `unit`, `integration`, `slow` for test categorization
- ****Config****: `asyncio\_mode = "auto"` in `pyproject.toml`

****Testing Environment****

- ****Conftest Bootstrapping****: Env var setup, config patching, LLM stubbing
- ****Unit Tests****: Fast, no external dependencies
- ****Integration Tests****: Docker-based (PostgreSQL, MongoDB)
- ****Coverage Tracking****: `just test-cov` for metrics

****Containerization****

- ****Docker Images****: Separate images per service
 - `Dockerfile\_agent\_server`: FastAPI + LangGraph
 - `Dockerfile\_mcp\_server`: FastMCP + PostgreSQL connector
 - `Dockerfile\_copilot\_runtime`: Node.js Express proxy

- ``Dockerfile_agent_server_debug``: With debugpy on port 5678
- ****Docker Compose****: Multi-service orchestration with volume mounts
- ****ECR Push****: AWS CodeBuild integration for CI/CD

****Package Management****

- ****uv**** (fast Python resolver): All Python dependencies managed via ``pyproject.toml``
- ****Workspace Dependencies****: ``database/`` as editable local package
- ****Dev Tool Groups****: ``test``, ``lint``, ``type`` groups for conditional installation
- ****Override Dependencies****: Langchain compatibility workarounds

****Build & Deployment****

- ****justfile****: 30+ recipes for dev/test/deploy workflows
- ****Commands****:
 - ``just install``: Python + database package
 - ``just install-copilot``: Node dependencies
 - ``just dev``: All 3 services with hot-reload
 - ``just test``: Unit tests
 - ``just typecheck``: mypy validation
 - ``just lint-fix``: Ruff auto-fix
 - ``just build-all``: Docker image build
 - ``just compose-up``: Docker Compose stack

**Configuration & Environment**

**Environment-Based Config**

- ****YAML Config**** (`config.yml`): Environment sections (dev, uat, prod, test, local)
- ****Env Vars****:
 - `ENV`: Active config section (dev/prod/test)
 - `LOG\_LEVEL`: Logging verbosity (INFO/DEBUG)
 - `LLM\_BACKEND`: LLM provider (openai/groq)
 - `CHECKPOINT\_BACKEND`: State storage (sqlite/postgres)
 - `PROMPT\_SOURCE`: Prompt location (langfuse/local/langfuse_with_fallback)
 - `MALA\_API\_URL`: Marketing platform API endpoint
 - `MALA\_MCP\_SERVER`: MCP server URL for tool routing
 - `SKIP\_AUTH`: Disable auth for local dev

**Secrets Management**

- AWS credentials for S3/DocumentDB
- OpenAI API key for LLM + embeddings
- Groq API key (fallback)
- Langfuse API keys for observability
- MongoDB TLS certificates (production)

**Performance Optimizations**

****Caching****

- ****MCP Tool Cache****: TTL-based to reduce redundant API calls
- ****KPI Embeddings Cache****: Pre-computed + lazy-loaded on first request
- ****LLM Responses Cache****: Langfuse SDK automatic caching

****Async/Await****

- ****Non-blocking I/O****: All database + API calls fully async
- ****Concurrent Tool Execution****: Multiple MCP tools execute in parallel
- ****Stream Buffering****: CopilotKit streaming with minimal latency

****Database Optimization****

- ****Connection Pooling****: psycopg[pool] for PostgreSQL
- ****Indexing****: MongoDB indexes on thread_id, timestamps
- ****PGVector****: Fast approximate nearest neighbor search

****Lever Agent Speedup****

- ****Explicit Checkpoint Subgraph****: Reduce redundant campaign data fetches
- ****Prefetch Injection****: Campaign data loaded before agent invocation
- ****Synthetic Tool Messages****: Bypass real MCP calls when data cached

****Advanced Features****

****Real-Time State Snapshots****

- Partial state updates during multi-turn conversations
- Manual state emission for frontend progress tracking
- `manually\_emitted\_state` for UI-specific fields (progress bars, etc.)

****AG-UI Integration****

- ****ag-ui-langgraph**** (0.0.28): Visual agent debugger
- ****ag-ui-protocol****: Standardized agent protocol
- ****Streaming Events****: Tool calls, state changes, messages

****RAG (Retrieval-Augmented Generation)****

- ****Document Indexing****: PDF, Word, PowerPoint, Excel parsing
- ****PGVector Storage****: Dense embeddings + BM25 hybrid search
- ****Knowledge Base****: MEC (marketing effectiveness center) documents
- ****Semantic Retrieval****: Fetch relevant guidelines for campaigns

****Automatic State Sync****

- ****BlueprintUpdater****: Monitors agent state changes
- ****MongoDB Persistence****: Writes to thread documents
- ****Fields Tracked****: KPI, MDE, campaign selection, lever recommendations

****Graph Interrupts & Human-in-the-Loop****

- ****interrupt()****: Pause agent for user input

- ****Control Flow Exceptions****: GraphInterrupt, GraphBubbleUp
- ****Langfuse Filtering****: Suppress false-positive error traces

****MCP Tool Deep Dive: `get\_kpi\_suggestions`****

****Purpose & Use Case****

The `get\_kpi\_suggestions` tool is a hybrid semantic search engine that recommends relevant KPIs based on a user's campaign objective. It powers Stage 4 (KPI Discovery) by matching natural language goal descriptions against a KPI dictionary using both keyword matching and semantic similarity, then ranks results using Reciprocal Rank Fusion (RRF).

****When Called****: KPI agent invokes this tool when the user describes their optimization goal (e.g., "I want to maximize purchases" or "improve click-through rate").

****Function Signature****

```
```python
```

```
async def get_kpi_suggestions(
 query: str, # Natural language goal (e.g., "maximize conversions")
 min_similarity: float = 0.4, # Minimum RRF score threshold (0-1)
 top_k: int | None = None, # Max results to return (None = all above threshold)
 campaign_id: str = "", # Optional: filter to campaign-feasible KPIs
) -> dict:
 ...
```

### ### \*\*Input Parameters\*\*

Parameter	Type	Default	Description
-----------	------	---------	-------------

-----	-----	-----	-----
-------	-------	-------	-------

`query`	`str`	Required	User's natural language goal or KPI mention (e.g., "I want to drive more clicks")
---------	-------	----------	-----------------------------------------------------------------------------------

`min_similarity`	`float`	0.4	Minimum RRF score (0-1 scale). Higher = stricter filtering. 0.4 filters out low-relevance noise.
------------------	---------	-----	--------------------------------------------------------------------------------------------------

`top_k`	`int` \ `None`	None	Limit results to top K candidates. None returns all above threshold.
---------	----------------	------	----------------------------------------------------------------------

`campaign_id`	`str`	""	Meta campaign ID for feasibility pre-filtering. When set, only KPIs with non-null data in MongoDB are returned.
---------------	-------	----	-----------------------------------------------------------------------------------------------------------------

### ### \*\*Return Value\*\*

```
```python
```

```
{  
    "suggestions": [  
        {  
            "sys_id": "purchase_roas",  
            "KPI_Metric": "Purchase ROAS",  
            "description": "Revenue per dollar spent on ads",  
            "KPI_type": "Effectiveness",  
            "KPI_category": "Revenue",  
            "KPI_subcategory": "Returns",  
            "funnel_stage": ["consideration", "conversion"],  
            "metric_family": "revenue_metrics"
```

```

    },
    # ... more KPI objects, ranked by relevance

]
}
...

```

**Algorithm: Hybrid Retrieval with RRF**

**Step 1: Initialize Cached KPI Corpus**

...

1. Check if KPI embeddings are cached in memory

└─ If yes: use cached vectors + tokenized texts

└─ If no: fetch KPI dictionary from MongoDB + compute embeddings (one-time cost)

2. Cache includes:

└─ documents: full KPI metadata (sys_id, name, description, etc.)

└─ embeddings: dense vectors from OpenAI text-embedding-3-large

└─ tokenized_texts: lowercase split for BM25 matching

└─ embeddings_model: OpenAI embedding encoder (reused across calls)

...

**Step 2: Optional Campaign Feasibility Filtering (when `campaign\_id` provided)**

...

1. Query MongoDB collections in parallel:

- └─ live_flatteneddatadump (recent campaign data)

- └─ flatteneddatadump (historical campaign data)

2. Projection: fetch only campaign_id + all KPI sys_ids

3. Merge results (live overrides historical):

- └─ Use latest value for each KPI column

- └─ Build set of "feasible_sys_ids" with non-null, non-empty data

4. Mask the corpus:

- └─ Keep only docs where sys_id ∈ feasible_sys_ids

- └─ Correspondingly mask embeddings and tokenized_texts arrays

- └─ If no feasible KPIs: return empty suggestions immediately

Example:

Input campaign_id: "123456"

Live data: {purchase_roas: 1.5, add_to_cart: 0.3}

Historical: {purchase_roas: 1.2, add_to_cart: 0.2, purchases: 100}

→ feasible_sys_ids: {purchase_roas, add_to_cart, purchases}

...

**Step 3: BM25 Keyword Scoring (Sparse Retrieval)**

...

1. Initialize BM25Okapi with tokenized KPI corpus

└ BM25 uses term frequency + inverse document frequency + document length normalization

2. Score each KPI against tokenized query:

user_query = "maximize purchase revenue"

tokens = ["maximize", "purchase", "revenue"]

Scores:

└ KPI "purchase_roas" → high score (matches "purchase", "revenue" in description)

└ KPI "add_to_cart_rate" → medium score (matches "revenue" intent weakly)

└ KPI "impressions" → low score (no token overlap)

3. Rank by BM25 score (descending):

└ bm25_rank[doc_idx] = 1 (best), 2, 3, ..., n (worst)

└ Example: {0: 1, 5: 2, 2: 3, ...}

...

Step 4: Vector Semantic Scoring (Dense Retrieval)

...

1. Embed the query using OpenAI text-embedding-3-large:

query_vec = embed("maximize purchase revenue")

→ 3072-dimensional vector

2. Compute cosine similarity between query_vec and all cached KPI embeddings:

similarities = cosine_similarity([query_vec], all_kpi_embeddings)

→ array of [0.0, 1.0] scores per KPI

3. Rank by similarity (descending):

└─ vec_rank[doc_idx] = 1 (most similar), 2, 3, ..., n

└─ Example: {0: 1, 3: 2, 5: 3, ...}

Example similarities:

KPI "purchase_roas": 0.89 (rank 1, very similar semantically)

KPI "revenue_per_click": 0.82 (rank 2)

KPI "impressions": 0.31 (rank 20, semantically dissimilar)

...

**Step 5: RRF Fusion (Reciprocal Rank Fusion)**

...

Combine BM25 and semantic rankings into single score:

$$\text{RRF_score}[i] = \text{BM25_WEIGHT} / (\text{RRF_CONSTANT} + \text{bm25_rank}[i]) \\ + \text{VECTOR_WEIGHT} / (\text{RRF_CONSTANT} + \text{vec_rank}[i])$$

Config (hardcoded):

BM25_WEIGHT = 0.4 # Give keyword matching 40% weight

VECTOR_WEIGHT = 0.6 # Give semantic similarity 60% weight

RRF_CONSTANT = 60 # Smoothing constant (prevents division by small numbers)

MAX_POSSIBLE_SCORE = 0.01 # Normalizing constant for 0-1 scaling

Normalize to 0-1 scale:

$\text{normalized_rrf}[i] = \text{rrf_scores}[i] / \text{MAX_POSSIBLE_SCORE}$

Example:

KPI with $\text{bm25_rank}=2$, $\text{vec_rank}=1$:

$\text{rrf} = 0.4/(60+2) + 0.6/(60+1) = 0.0059 + 0.0095 = 0.0154$

$\text{normalized} = 0.0154 / 0.01 = 1.54$ (capped at threshold)

KPI with $\text{bm25_rank}=50$, $\text{vec_rank}=45$:

$\text{rrf} = 0.4/(60+50) + 0.6/(60+45) = 0.00222 + 0.00444 = 0.00666$

$\text{normalized} = 0.00666 / 0.01 = 0.67$ (likely filtered out by $\text{min_similarity}=0.4$)

...

Step 6: Threshold Filtering

...

1. Filter: keep only KPIs where $\text{normalized_rrf} \geq \text{min_similarity}$

└─ $\text{min_similarity}=0.4$ (default): mid-range relevance + above

└─ $\text{min_similarity}=0.8$: only highest-relevance matches

└─ Returns empty if no candidates exceed threshold

2. Sort descending by RRF score:

Candidates = [KPI_1 (0.92), KPI_2 (0.78), KPI_5 (0.65), ...]

...

Step 7: Family & Category Exclusion

...

Remove KPIs that don't apply to Split Test experiments:

Excluded metric_family:

- └ "brand_lift" (only for brand lift studies, not split tests)
- └ "conversion_lift" (only for conversion lift, not split tests)
- └ "estimated_ad_recall" (brand measurement, not split test metric)

Excluded KPI_subcategory:

- └ "Branding" (brand awareness, not testable in A/B test)
- └ "Conversions" (redundant with conversion_lift, context-dependent)

Example exclusion:

Input: [purchase_roas, brand_awareness, add_to_cart_rate, estimated_reach]

After filter: [purchase_roas, add_to_cart_rate]

(brand_awareness excluded, estimated_reach excluded)

...

Step 8: Top-K Truncation

...

If top_k is specified, return only first K candidates:

top_k=3: return [KPI_1, KPI_2, KPI_5] (drop all others)

top_k=None: return all passing threshold

Example (top_k=3):

Input: [purchase_roas (0.92), add_to_cart_rate (0.78), ctr (0.65), cpm (0.55)]

Output: [purchase_roas, add_to_cart_rate, ctr]

...

Caching Strategy

KPI Embedding Cache (`\_ensure\_kpi\_embeddings`)

...

Initialization (first request):

1. Fetch full KPI_DICTIONARY from MongoDB

2. Build text corpus from 8 fields per KPI:

{sys_id, KPI_Metric, description, KPI_type, category, subcategory, funnel_stage, metric_family}

3. Tokenize all texts

4. Call OpenAI embeddings API once for entire corpus (~1000 KPIs)

5. Store in module-level variable: _kpi_embeddings_cache

Cost: ~2-5 seconds + OpenAI API token cost (~\$0.01)

Subsequent requests:

└─ Return cached embeddings immediately (negligible latency)

└ No re-computation unless code redeploys or cache invalidated

Thread safety:

└ Async lock (`_kpi_cache_lock`) prevents multiple concurrent initialization attempts

└ Double-check after lock acquisition ensures single initialization even under load

...

Error Handling

```
```python
```

```
try:
```

```
 # ... all above steps
```

```
except Exception as e:
```

```
 logger.exception("get_kpi_suggestions failed: %s", e)
```

```
 return {"status": "error", "message": str(e)}
```

```
...
```

Returns `{"status": "error", "message": "..."}`  instead of raising, allowing graceful fallback.

### ### \*\*Performance Characteristics\*\*

Scenario	Latency	Notes
----------	---------	-------

-----	-----	-----
-------	-------	-------

First request (cache init)	2-5 seconds	One-time embedding computation
----------------------------	-------------	--------------------------------

Subsequent requests (cached)	50-200ms	BM25 + cosine similarity + RRF on CPU
With campaign\_id filter	+50-100ms	Parallel MongoDB queries
Large top\_k (500+)	+10-20ms	Array slicing overhead minimal

### ### **\*\*Usage Examples\*\***

#### #### **\*\*Example 1: Basic KPI Search\*\***

```
```python
await get_kpi_suggestions(
    query="I want to drive more purchases",
    min_similarity=0.4,
    top_k=5
)
```
```

#### **\*\*Expected output\*\*:**

```
```json
{
  "suggestions": [
    {
      "sys_id": "purchase_roas",
      "KPI_Metric": "Purchase ROAS",
      "KPI_type": "Effectiveness",
      ...
    }
  ]
}
```

```

    },
    {
        "sys_id": "purchase_rate",
        "KPI_Metric": "Purchase Rate",
        "KPI_type": "Volume",
        ...
    },
    ...
]
}
...

```

****Example 2: Campaign-Specific KPI Suggestions****

```

```python
await get_kpi_suggestions(
 query="optimize click-through rate",
 campaign_id="123456", # Only return KPIs with data for this campaign
 min_similarity=0.5,
 top_k=10
)
...

```

**\*\*Behavior\*\*:**

1. Fetches campaign 123456's KPI data from MongoDB

2. Filters KPI corpus to only those with recorded values
3. Runs hybrid search on filtered set
4. Returns top 10 relevant KPIs with campaign data present

#### #### **\*\*Example 3: Strict Relevance Filtering\*\***

```
```python
await get_kpi_suggestions(
    query="reduce cost per acquisition",
    min_similarity=0.75 # Only high-confidence matches
)
```
```

**\*\*Effect\*\***: Only KPIs with strong keyword + semantic alignment returned (fewer false positives).

#### ### **\*\*Integration with KPI Agent\*\***

The KPI agent calls `get\_kpi\_suggestions` when:

1. **\*\*User describes goal\*\***: "I want to maximize revenue per user"
2. **\*\*Agent processes request\*\***:
  - Calls `get\_kpi\_suggestions(query="maximize revenue per user")`
  - Receives ranked list of candidate KPIs
  - Presents top 3-5 options to user via UI
3. **\*\*User selects KPI\*\***: Agent stores selection + proceeds to power calculation

### ### \*\*Key Design Decisions\*\*

| Decision | Rationale |

|-----|-----|

| **\*\*Hybrid BM25 + Vector\*\*** | BM25 catches exact metric name matches; vector catches semantic intent. Combined = robust. |

| **\*\*RRF Fusion\*\*** | Avoids tuning linear combination weights; reciprocal rank emphasizes agreement across rankers. |

| **\*\*0.4 min\_similarity default\*\*** | Filters obvious noise while preserving borderline-relevant suggestions for diversity. |

| **\*\*Campaign feasibility filter\*\*** | Prevents recommending KPIs with zero data (user frustration when data unavailable). |

| **\*\*Exclude brand\_lift/conversion\_lift\*\*** | These are for different test types; split test agents shouldn't see them. |

| **\*\*Eager embedding cache\*\*** | One-time OpenAI cost paid on first request; thereafter sub-100ms latency. |

### ### \*\*Limitations & Future Improvements\*\*

#### **\*\*Current Limitations\*\*:**

- Does not learn from user click-through behavior (recommendations static)
- Campaign feasibility check requires all MongoDB documents (slow for sparse data)
- No disambiguation when query mentions multiple KPI types

#### **\*\*Potential Enhancements\*\*:**

- Ranking feedback loop: track user-selected KPIs → retrain RRF weights
- Approximate nearest neighbor (ANN) for sub-10ms dense search at scale

- Multi-language support (currently English-only embeddings)
- Query expansion: "maximize revenue" → ["revenue", "roas", "profit", "ltv"]

---

## ## \*\*Project Structure\*\*

...

spi-mforce-BE/

```

├── mforce/ # Agent Server
│ ├── agent_server.py # FastAPI app + CopilotKit handler
│ ├── agents/
│ │ ├── mforce.py # Main orchestrator (6-stage workflow)
│ │ ├── prerequisite_workflow.py
│ │ ├── objective_agent.py
│ │ ├── test_agent.py
│ │ ├── kpi/agent.py # KPI selection with structured output
│ │ ├── lever_agent/ # Campaign analysis & optimization
│ │ └── experiment_design_workflow.py
│ ├── common.py # BlueprintUpdater, state sync
│ ├── mcp_connector.py # MCP tool cache & client
│ └── langfuse_connector.py # Prompt management
├── api/threads.py # Thread CRUD endpoints
└── common/
```

```
| | └─ types.py # Pydantic models (KPIContext, WorkflowContext)
| | └─ dependencies.py # FastAPI deps
| └─ prompts/ # Local fallback prompt files
| └─ memory_stores/
| | └─ sqlite.py # SQLite checkpointer
| | └─ postgres.py # PostgreSQL checkpointer
| └─ config.yml # Environment-based config
| └─ utils.py # LLM factory, config loader
|
└─ mcp/ # MCP Server
 └─ mcp_server.py # FastMCP + async handlers
 └─ tools/
 └─ functions.py # Tool implementations
 └─ mcp_tool_*.py # 20+ individual tools
 └─ pg_vectorstore.py # PGVector hybrid search
 └─ local_prompt_loader.py
 └─ aws_utils.py # S3 + boto3 integration
 └─ requirements.txt
|
└─ database/ # Shared Database Package
 └─ database/
 └─ __init__.py
 └─ collections.py # MongoDB collection names
 └─ services/
```

```

| | └─ service.py # BaseMongoRepository, ThreadService
| └─ pyproject.toml
|
└─ copilot-runtime/ # Node.js Proxy (optional)
 └─ runtime.js
 └─ package.json
 └─ node_modules/
 |
 └─ tests/
 └─ conftest.py # Pytest bootstrapping
 └─ unit/ # Unit tests
 └─ integration/ # Integration tests (Docker)
 |
 └─ docker-compose.yml # Multi-service orchestration
 └─ Dockerfile_* # Service-specific images
 └─ buildspec.yml # AWS CodeBuild pipeline
 └─ justfile # 30+ dev recipes
 └─ pyproject.toml # Dependencies + tool config
...

```

**## \*\*Development Workflow\*\***

### **### \*\*Local Setup\*\***

```
```bash
```

```
just install      # Install Python deps
```

```
just install-copilot # Install Node deps
```

```
just install-all  # Both
```

```
```
```

### **### \*\*Running Services\*\***

```
```bash
```

```
just dev          # Start all 3 services (hot-reload)
```

```
just dev-agent     # Agent server only
```

```
just dev-mcp       # MCP server only
```

```
just dev-copilot   # Copilot runtime only
```

```
just dev-agent-debug # Agent with debugpy on :5678
```

```
```
```

### **### \*\*Validation\*\***

```
```bash
```

```
just lint          # Ruff linting
```

```
just format-check  # Format validation
```

```
just typecheck     # mypy type checking
```

```
just check         # All three
```

```
just fix           # lint-fix + format
```

```
```
```

### ### \*\*Testing\*\*

```
```bash
```

```
just test      # Unit tests (pytest)
```

```
just test-integration # Integration tests (Docker required)
```

```
just test-cov      # Coverage report
```

```
just test-file <path> # Single file
```

```
just test-k <keyword> # Tests matching keyword
```

```
```
```

### ### \*\*CLI Pipeline\*\*

```
```bash
```

```
just cli-run      # Full 4-agent pipeline (headless)
```

```
just cli-agent lever # Single agent execution
```

```
just eval scenario session # LLM-as-a-Judge evaluation
```

```
```
```

```

```

### ## \*\*Challenges Solved\*\*

1. **Multi-Stage Agent Orchestration**: Implemented unified router for conditional stage transitions with resumption support
2. **Streaming State Snapshots**: Handled ag-ui-langgraph bugs (6 patches) for reliable frontend synchronization

3. **Statistical Power Calculations**: Accurate sample size + duration estimation across test types
4. **Hybrid Search for KPIs**: Combined BM25 + vector embeddings with RRF for optimal recall/precision
5. **Database Persistence**: MongoDB + PostgreSQL checkpointing for scalable multi-instance deployment
6. **Tool Integration**: Reliable MCP tool routing with TTL caching + forced completion gates
7. **Prompt Versioning**: Langfuse-backed centralized prompt management with local fallback
8. **Async Architecture**: Non-blocking I/O throughout for optimal latency under load

---

## ## **Deployment**

### ### **Local Development**

- Docker Compose stack with MongoDB, PostgreSQL, all 3 services
- Hot-reload for rapid iteration
- SQLite checkpointing (default)

### ### **Production**

- AWS ECR for image registry
- ECS/Fargate for container orchestration
- DocumentDB (MongoDB-compatible) + RDS PostgreSQL
- S3 for artifact storage
- Application Load Balancer for traffic routing

### ### **\*\*CI/CD\*\***

- **\*\*AWS CodeBuild\*\***: Triggers on git push
- **\*\*Build Steps\*\***: Docker builds, ECR push, image tagging
- **\*\*Pre-commit Hooks\*\***: Ruff, mypy, trailing whitespace validation

---

### ## **\*\*Scalability & Reliability\*\***

- **\*\*Async/Await\*\***: Non-blocking I/O for high throughput
- **\*\*Connection Pooling\*\***: Psycopg connection pool + boto3 session reuse
- **\*\*Checkpoint Persistence\*\***: PostgreSQL-backed multi-instance support
- **\*\*Error Recovery\*\***: Graceful tool failure handling + fallback mechanisms
- **\*\*Observability\*\***: Langfuse tracing + LangSmith debugging for production diagnostics
- **\*\*Rate Limiting\*\***: Built-in safeguards for LLM API calls + tool execution

---

### ## **\*\*Resume Summary\*\***

Built a sophisticated **\*\*multi-agent orchestration platform\*\*** using **\*\*LangGraph, FastAPI, and CopilotKit\*\*** that guides users through AI-driven marketing experiment design. Architected **\*\*3 microservices\*\*** with seamless **\*\*MCP tool integration\*\***, **\*\*hybrid semantic search\*\***, and **\*\*real-time conversational AI\*\***. Implemented **\*\*statistical power calculations\*\***, **\*\*state checkpointing\*\*** across PostgreSQL/MongoDB, and **\*\*Langfuse-backed prompt versioning\*\***. Achieved production-grade reliability through **\*\*comprehensive test coverage\*\*** (pytest), **\*\*strict code quality\*\*** (Ruff + mypy), and **\*\*AWS containerization\*\*** (ECR/Fargate). Leveraged **\*\*async/await\*\*** throughout for optimal performance and delivered a **\*\*feature-rich**

**agent platform\*\*** that handles complex multi-turn workflows with graceful error recovery and human-in-the-loop interrupts.

---

**\*\*Technologies\*\***: Python 3.12, FastAPI, LangGraph, LangChain, CopilotKit, MCP, OpenAI/Groq LLMs, MongoDB, PostgreSQL+PGVector, FastMCP, Pydantic, Ruff, mypy, pytest, Docker, AWS (ECR, Fargate, S3, DocumentDB, RDS), Langfuse, LangSmith, Node.js, Express